

RAG + LangChain Architecture & Design Guide

Production-oriented Retrieval-Augmented Generation system using
free Hugging Face libraries and tools

1. Overview & Core Concept

RAG (Retrieval-Augmented Generation) is not a single model — it is a pipeline made of independent stages. The design principle underlying this document: treat **ingestion** and **query/generation** as two separate systems that can evolve, scale, and fail independently.

```
User Query --> Embed Query --> Search Vector DB --> Retrieve Top-K Chunks -->
Build Prompt (Context + Query) --> LLM Generates Answer --> Return + Citations
```

Design Principle: Ingestion (offline/batch) and Query (online/real-time) pipelines must be decoupled. Building RAG as a single monolithic script is the most common mistake in early-stage implementations and blocks scaling later.

1.1 High-Level System Design



1.2 Component Map (Free Hugging Face Stack)

Component	Purpose	Free Tool / Library
Document Loader	Read PDFs / docs / web / CSV	langchain_community.document_loaders
Text Splitter	Chunk long text	langchain.text_splitter (RecursiveCharacterTextSplitter)
Embedding Model	Text → vectors	sentence-transformers/all-MiniLM-L6-v2 via langchain-huggingface
Vector Store	Store & search vectors	FAISS (local) or ChromaDB (local, multi-tenant)
Retriever	Fetch relevant chunks	LangChain VectorStoreRetriever
LLM	Generate final answer	HF Inference API (free tier) — e.g. Mistral-7B-Instruct
Orchestration	Wire pipeline together	LangChain / LangGraph (LCEL chains)

2. Folder Structure (Production-Style)

This structure separates concerns so any single piece (vector store, embedding model, LLM provider) can be swapped without touching the rest of the system.

```
rag-project/
├─ app/
│  ├── main.py                # FastAPI entrypoint
│  ├── config.py              # settings via pydantic-settings
│  │
│  └─ ingestion/
│     ├── loaders/            # registry + per-type loaders
│     │  ├── registry.py
│     │  ├── pdf_loader.py
│     │  ├── web_loader.py
│     │  ├── docx_loader.py
│     │  ├── csv_loader.py
│     │  └─ db_loader.py
│     ├── splitter.py
│     ├── embedder.py
│     └─ vectorstore_builder.py # tenant-aware
│
│  └─ retrieval/
│     ├── retriever.py
│     ├── query_expander.py    # optional: HyDE / multi-query
│     └─ reranker.py           # optional cross-encoder rerank
│
│  └─ generation/
│     ├── llm_provider.py      # HF now, swappable later
│     ├── prompt_templates.py
│     └─ chain.py              # LangChain RAG chain (LCEL)
│
│  └─ api/
│     ├── routes_ingest.py     # POST /documents
│     ├── routes_query.py      # POST /chat
│     └─ deps.py               # auth, tenant resolution
│
│  └─ utils/
│     ├── resilient_call.py    # retry / backoff wrapper
│     ├── cache.py             # query-response cache
│     └─ logger.py
│
│  └─ schemas/
│     └─ models.py             # Pydantic request/response models
│
├─ data/
│  ├── raw/                    # original documents
│  ├── processed/              # cleaned / chunked text
│  └─ vectorstore/{tenant_id}/ # per-tenant persisted index
│
├─ tests/
│  ├── test_ingestion.py
│  ├── test_retrieval.py
│  └─ test_generation.py
│
├─ notebooks/                  # experimentation only
│  └─ prototype_rag.ipynb
│
├─ scripts/
│  ├── build_index.py          # CLI: run ingestion pipeline
│  └─ evaluate_rag.py          # RAGAS or custom evaluation
│
├─ requirements.txt
├─ .env.example
├─ Dockerfile
└─ README.md
```

Why this shape: Ingestion, retrieval, and generation are isolated modules. You can move from FAISS to Pinecone, or from MiniLM to a larger embedding model, by editing one file — never the pipeline logic that depends on it.

3. Implementation — Core Modules

3.1 Embedding Layer (local, free, no rate limits)

```
# app/ingestion/embedder.py
from langchain_huggingface import HuggingFaceEmbeddings

def get_embedder():
    return HuggingFaceEmbeddings(
        model_name="sentence-transformers/all-MiniLM-L6-v2"
    )
```

3.2 Multi-Source Ingestion — Loader Registry

A SaaS product ingesting mixed sources (PDFs, web pages, structured DB rows, CSVs) needs a registry pattern rather than one hardcoded loader.

```
# app/ingestion/loaders/registry.py
from langchain_community.document_loaders import (
    PyPDFLoader, WebBaseLoader, Docx2txtLoader, CSVLoader
)

LOADER_MAP = {
    ".pdf": PyPDFLoader,
    ".docx": Docx2txtLoader,
    ".csv": CSVLoader,
}

def load_document(source: str, source_type: str):
    if source_type == "web":
        return WebBaseLoader(source).load()
    ext = source.split(".")[-1]
    loader_cls = LOADER_MAP.get(f".{ext}")
    if not loader_cls:
        raise ValueError(f"No loader registered for {ext}")
    return loader_cls(source).load()
```

Tag every document with metadata at ingestion time: `source_type`, `upload_date`, `tenant_id`. This is required later for per-customer data isolation and debugging.

3.3 Vector Store — Multi-Tenant (ChromaDB)

For a production SaaS product, avoid one flat FAISS index shared across all customers. ChromaDB supports per-tenant collections and metadata filtering natively.

```
# app/ingestion/vectorstore_builder.py
import chromadb
from langchain_chroma import Chroma
from app.ingestion.embedder import get_embedder

def get_vectorstore(tenant_id: str):
    client = chromadb.PersistentClient(path=f"data/vectorstore/{tenant_id}")
    return Chroma(
        client=client,
        collection_name=f"tenant_{tenant_id}",
        embedding_function=get_embedder(),
    )
```

3.4 Ingestion Pipeline (FAISS variant, single-tenant / prototype)

```
# app/ingestion/vectorstore_builder.py (FAISS variant)
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
from app.ingestion.embedder import get_embedder

def build_index(pdf_path: str, save_path: str = "data/vectorstore"):
    docs = PyPDFLoader(pdf_path).load()
    splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
    chunks = splitter.split_documents(docs)

    embeddings = get_embedder()
    vectorstore = FAISS.from_documents(chunks, embeddings)
    vectorstore.save_local(save_path)
    return vectorstore
```

3.5 LLM Provider Abstraction (HF Inference API, free tier)

Important: The free Hugging Face Inference API tier is not production-safe long term — rate limits, cold starts, and no SLA. Build the LLM behind a provider abstraction now so upgrading to a paid Inference Endpoint later is a config change, not a rewrite.

```
# app/generation/llm_provider.py
from langchain_huggingface import HuggingFaceEndpoint
from app.config import settings

def get_llm():
    """
    Provider abstraction — swap via config, not code changes.
    Free tier now; paid Inference Endpoint later = same interface.
    """
    return HuggingFaceEndpoint(
        repo_id=settings.LLM_MODEL,          # e.g. "mistralai/Mistral-7B-Instruct-v0.3"
        huggingfacehub_api_token=settings.HF_TOKEN,
        temperature=0.2,
        max_new_tokens=512,
        timeout=30,
    )
```

3.6 Resilience — Retry & Backoff Wrapper

```
# app/utils/resilient_call.py
import time
from functools import wraps

def with_retry(max_attempts=3, backoff=2):
    def decorator(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            for attempt in range(max_attempts):
                try:
                    return fn(*args, **kwargs)
                except Exception as e:
                    if attempt == max_attempts - 1:
                        raise
                    time.sleep(backoff ** attempt)
            return wrapper
        return decorator
```

3.7 RAG Chain — Full Query Pipeline (LangChain LCEL)

```

# app/generation/chain.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser
from langchain_community.vectorstores import FAISS
from app.ingestion.embedder import get_embedder
from app.generation.llm_provider import get_llm

def build_rag_chain():
    vs = FAISS.load_local("data/vectorstore", get_embedder(),
                          allow_dangerous_deserialization=True)
    retriever = vs.as_retriever(search_kwargs={"k": 4})

    prompt = ChatPromptTemplate.from_template("""
Answer using only the context below. If unsure, say you don't know.

Context: {context}
Question: {question}
""")

    def format_docs(docs):
        return "\n\n".join(d.page_content for d in docs)

    chain = (
        {"context": retriever | format_docs, "question": RunnablePassthrough()}
        | prompt
        | get_llm()
        | StrOutputParser()
    )
    return chain

```

4. Production Design Considerations (SaaS)

4.1 Multi-Tenancy

Every document ingested must carry a `tenant_id`. Vector store collections, cache keys, and logs should all be namespaced by tenant to prevent data leakage between customers.

4.2 Rate Limits & Cold Starts (Free HF Inference API)

- Wrap every LLM call in retry + exponential backoff (see 3.6)
- Cache repeated queries — identical questions should not re-hit the API
- Set explicit timeouts; do not let a hung request block the user-facing request thread
- Plan the migration path to a paid Inference Endpoint or self-hosted TGI before real user traffic arrives

4.3 Evaluation Before Launch

Use **RAGAS** (or a custom evaluation script) to score faithfulness, answer relevance, and context precision before shipping. Log every query with its retrieved chunks and final answer — this is the only reliable way to debug hallucinated or wrong answers after launch.

4.4 Data Flow Summary

Stage	Trigger	Output
Ingestion	New document uploaded	Chunks embedded & stored in tenant's vector collection
Query	User sends a chat message	Retrieved context + generated answer
Evaluation	Scheduled / pre-release	Faithfulness & relevance scores per query set

5. Minimum Viable Production Checklist

- ☐ Provider abstraction so LLM / embedding source is config-driven, not hardcoded
- ☐ Retry + timeout handling around every Hugging Face API call
- ☐ Response caching to avoid repeat cost and latency on repeated queries
- ☐ Per-tenant data isolation in the vector store
- ☐ Basic RAG evaluation (RAGAS: faithfulness, answer relevance) completed before launch
- ☐ Logging that captures query → retrieved chunks → final answer for debugging
- ☐ A defined upgrade path for when free-tier rate limits are hit in production

Manager's Note: The free Hugging Face Inference API is genuinely fine to launch on for early traction, but budget for a paid endpoint or self-hosted model before real concurrent traffic arrives — cold starts and shared rate limits will directly hurt user experience at that point.